Database Design Document

RSCI

RSCI FYP Portal

Database Design Document

Final Year Project Management Portal

Riphah School of Computing & Innovation

DOCUMENT VERSION
1.0

DATE
14 October 2026

TARGET DATABASE
PostgreSQL 15+

STATUS
Developer-ready baseline

Companion to SRS v1.0 and SDD v1.0

Document control

TITLE
RSCI FYP Portal Database Design Document

VERSION
1.0

DATE
14 October 2026

SYSTEM
RSCI FYP Portal

OWNER
Riphah School of Computing & Innovation

TARGET RDBMS
PostgreSQL 15+

CLASSIFICATION
Internal project documentation

Companion documents

Document	Version	Artifact slug
Software Requirements Specification	1.0	rsci-fyp-portal-srs
System Design Document	1.0	rsci-fyp-portal-sdd
FYP Framework	1.0	FYP_Framework_v1.0-updated.pdf

Approval record

Role	Name	Decision	Date
FYP Coordinator	To be assigned	Pending	—
Head of Department	To be assigned	Pending	—
Technical Lead	To be assigned	Pending	—

Status. Committee-owned policies and default weightages are explicitly marked configurable; this document does not convert proposed values into institutional policy.

Table of contents

1. Introduction and conventions 4
2. Entity-relationship overview 6
3. Security and authorization model 8
4. Full table specifications 10
5. Enum registry and state controls 45
6. Integrity, triggers, and group-code generation 46
7. Indexing and transaction design 49
8. Migration and seed plan 51
9. Reporting, backup, and testing 54
10. Starter DDL 57
11. Traceability and open issues 60

How to use this document

Developers should treat the table specifications, constraints, transaction boundaries, and starter DDL as one contract. Product owners should review every item labelled *configurable* or *open decision*. Database migrations must preserve historical scores, submissions, minutes, and audit records.

Purpose, scope, and references

Purpose

This document defines the relational persistence model for the RSCI FYP Portal. It turns the functional lifecycle into tables, keys, constraints, indexes, triggers, state transitions, seed data, migration rules, and operational controls that a development team can implement.

Scope

The model covers identities and role grants; academic sessions and programs; ten FYP project types; group formation; proposals and reviews; supervision; meetings, attendance, minutes and actions; milestones and versioned deliverables; AI declarations; staged evaluations, panels, rubrics and marks; notifications; announcements; audit evidence; and per-student effort records.

Source references

Reference	Use
SRS v1.0	Functional and non-functional contract
SDD v1.0	Application architecture and integration contract
RSCI FYP Framework v1.0	Project types, suitable programs, common deliverables, AI rules, 9 hours/week and 300-hour expectation
PostgreSQL 15+	Target SQL dialect and database features

Out of scope

Binary file storage, email delivery, identity-provider internals, source-code hosting, video conferencing, and analytics warehouse implementation are external services. The database stores their references and processing state.

Conventions and implementation baseline

Naming and types

Area	Convention
Tables/columns	snake_case; plural table names
Primary keys	id; uuid unless a small static catalogue
Foreign keys	_id
Time	timestamptz in UTC; dates only for academic deadlines
Audit times	created_at and updated_at
Deletion	Prefer deactivate/archive; deleted_at only where approved
Enums	PostgreSQL enum or checked lookup; migration-reviewed
Files	Object key or validated URL; never database BLOB
Money/marks/hours	numeric with explicit precision; never float

Required extensions

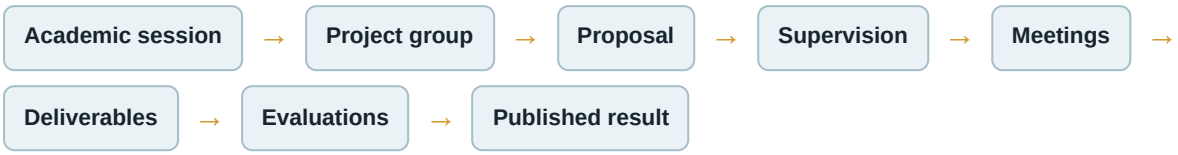
`pgcrypto` provides `gen_random_uuid()`; `citext` provides case-insensitive email uniqueness. `pg_trgm` is optional for fuzzy title search.

Baseline decisions

- Use UUIDs at trust boundaries and for all workflow entities; use smallserial only for roles and permissions.
- Use UTC in storage and convert to the user or campus timezone in the application.
- Use RESTRICT for academic catalogue/history dependencies, CASCADE for true children, and SET NULL only where preserving evidence is more important than actor retention.
- Use transactions for every workflow transition; do not let API clients write status fields directly.
- Use optimistic concurrency with updated_at or an added row_version column on heavily edited records.

> **Configurable.** Supervision load, group size, stage dates, weightages, rubric content, grace periods, and retention periods require committee approval.

The lifecycle has one auditable ownership spine



The academic session scopes programs, role grants, milestones, and evaluation stages. A project group belongs to exactly one session and one project type. Students join through memberships; supervisors join through allocations. A group owns one current proposal, many meeting records, versioned deliverable submissions, and one evaluation per configured stage. Scores are evaluator-by-criterion facts; weighted results are derived, moderated, then published.

Ownership rules

- A child record inherits the group session; cross-session references are rejected by triggers.
- Project type controls applicable deliverables, rubric override, and AI policy.
- Proposal, meeting, submission, declaration, review, and score history is retained rather than overwritten silently.
- Published or signed academic records are frozen and reopened only through an audited privileged operation.
- Results are calculated from stage weightage, rubric weightage, criterion maximum, and panel aggregation; calculations must be deterministic and versioned.

Result persistence. A published result may be exposed through a view. If signed result snapshots are legally required, add result_publications with calculation_version, total, breakdown JSON, published_by, and published_at.

Relationship summary

Entity	Related entities	Cardinality
academic_sessions	session_programs, groups, milestones, stages	1:N
users	roles, memberships, reviews, panels, scores	1:N
project_types	groups, milestones, templates, rubrics	1:N
project_groups	members, proposals, meetings, submissions, evaluations	1:N
proposals	proposal_reviews	1:N
meetings	attendance, minutes, action_items	1:N / 1:1
milestones	deliverable_templates	1:N
deliverable_templates	deliverable_submissions	1:N
deliverable_submissions	submission_reviews, ai_declarations	1:N
evaluation_stages	rubrics, evaluations	1:N
rubrics	rubric_criteria	1:N
evaluations	panels, scores	1:N

Deletion semantics

Academic history is not physically deleted in normal operation. Cascades are limited to dependent rows that have no meaning outside the parent, such as meeting attendance. Users referenced by formal decisions are normally deactivated; reviewer/evaluator foreign keys use `ON DELETE RESTRICT`. Retention-driven anonymization requires a separately approved migration.

Polymorphic link exception

`notifications.related_entity_type/id` and `audit_logs.entity_type/id` cannot use ordinary foreign keys. The application must validate allowed entity types, and scheduled integrity checks should identify orphaned links. All academic facts continue to use real foreign keys.

Authorization is enforced at both service and database layers

Authorization model

Roles are grants; permissions are capabilities. Global roles use a null session in `user_roles`; operational roles are session-scoped. The API resolves effective permissions from global and matching-session grants, then passes `app.user_id` and `app.session_ids` into the database transaction.

Suggested row-level security policies

Data	Read	Write
Own profile	Self; authorized admins	Self-limited; admins
Group records	Members, allocations, authorized evaluators/coordinators	Owner role by workflow state
Meetings	Group members and active supervisors	Organizer/supervisor; signed minutes frozen
Submissions	Group, reviewers, coordinators	Group submits; reviewers decide
Evaluations	Panel, group after publication, coordinators	Panel scores; chair/coordinator moderates
Audit logs	Security/admin and approved auditors	Database trigger only

Database roles

- portal_migrator owns schema changes and is never used by the web application.
- portal_app executes approved CRUD and functions but cannot disable triggers or edit audit rows.
- portal_readonly is used for support/reporting views with masked personal data.
- portal_worker handles outbox/notification tasks with narrow grants.

> **Critical.** Do not rely on hidden buttons for authorization. Every mutation path must re-check effective role, session scope, project ownership, workflow state, and conflict-of-interest rules.

Privacy, audit, and file-reference controls

Sensitive data handling

- Hash passwords only in the identity service with Argon2id or a current approved algorithm; store only the encoded hash.
- Encrypt backups and storage; use TLS for all database connections.
- Redact password hashes, tokens, private file URLs, phone numbers, and full document bodies from audit JSON.
- Use signed, short-lived download URLs generated by the file service; store stable object keys where possible.
- Limit external examiner and industry partner access to assigned groups and explicitly released artifacts.
- Keep IP addresses only for an approved security-retention period.

Audit event minimums

Action class	Examples
Identity/authorization	role grant/revoke, login outcome, deactivate user
Academic workflow	proposal decision, allocation change, meeting sign-off
Artifacts	submission, review, acceptance, declaration attestation
Evaluation	panel change, score edit, moderation, publish/reopen
Policy/configuration	project type, milestone, stage, rubric, weightage
Operations	migration, import, export, retention/anonymization

Append-only. The application role receives INSERT through controlled functions only and no UPDATE/DELETE on audit_logs. Consider monthly partitions and a cryptographic chain if tamper-evident records are required.

Full table specifications

Reading the specifications

Each table page lists DDL-ready columns and the minimum database controls. “Null” describes storage nullability, not whether a UI field appears optional. Every foreign-key column receives a btree index unless a listed composite index already has that column as its leading key.

Common rules

- UUID primary keys default to gen_random_uuid().
- All timestamps are timestamptz and stored in UTC.
- All text input is trimmed; empty strings are rejected where a value is required.
- updated_at is changed by a shared trigger, not trusted from the client.
- Workflow changes go through stored functions or a service transaction that also writes audit_logs.
- DDL migration names use a monotonic version and a descriptive suffix.

> **Versioning.** Deliverable submissions and AI declarations are immutable versions. Proposals use one current row with a version counter plus audit evidence; add a proposal_versions child table if full byte-for-byte proposal snapshots are required.

users

> Stores the canonical login identity and profile shared by every portal actor.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
email	citext	No	—	Normalized, case-insensitive login
password_hash	text	Yes	NULL	Null only for federated login
full_name	varchar(200)	No	—	Display and formal name
phone	varchar(32)	Yes	NULL	Store E.164 where available
avatar_url	text	Yes	NULL	Object-storage URL/key
is_active	boolean	No	true	Administrative access switch
last_login_at	timestamptz	Yes	NULL	Successful login time
created_at	timestamptz	No	now()	Creation time
updated_at	timestamptz	No	now()	Last mutation time

Primary key

- id

Foreign keys

- None

Unique constraints

- email

Indexes

- UNIQUE btree (email)
- btree (is_active) WHERE is_active = true

Check constraints

- email <> ''
- phone IS NULL OR length(phone) BETWEEN 7 AND 32

Triggers / guards

- set_updated_at before update
- reject blank full_name

roles

> Defines the fixed authorization roles that can be granted globally or within an academic session.

Column	Type	Null	Default	Notes
id	smallserial	No	sequence	Primary identifier
name	varchar(40)	No	—	Seeded role code
description	text	Yes	NULL	Human-readable scope

Primary key

- id

Foreign keys

- None

Unique constraints

- name

Indexes

- UNIQUE btree (name)

Check constraints

- name IN (SUPER_ADMIN, FYP_COORDINATOR, HOD, SUPERVISOR, CO_SUPERVISOR, STUDENT, INTERNAL_EVALUATOR, EXTERNAL_EXAMINER, INDUSTRY_PARTNER)

Triggers / guards

- None

permissions

> Defines atomic capabilities used by role-based access control.

Column	Type	Null	Default	Notes
id	smallserial	No	sequence	Primary identifier
code	varchar(100)	No	–	Verb-oriented code, e.g. groups.approve
description	text	No	–	What the permission authorizes

Primary key
- id

Foreign keys
- None

Unique constraints
- code

Indexes
- UNIQUE btree (code)

Check constraints
- code ~ '^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$'

Triggers / guards
- None

role_permissions

> Maps roles to atomic permissions.

Column	Type	Null	Default	Notes
role_id	smallint	No	–	Role
permission_id	smallint	No	–	Permission

Primary key

- (role_id, permission_id)

Foreign keys

- role_id → roles.id ON DELETE CASCADE
- permission_id → permissions.id ON DELETE CASCADE

Unique constraints

- None beyond primary key

Indexes

- PK supports role lookup
- btree (permission_id) for reverse lookup

Check constraints

- None

Triggers / guards

- None

user_roles

> Grants a role to a user globally or for one academic session.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Grant identifier
user_id	uuid	No	—	Grantee
role_id	smallint	No	—	Granted role
session_id	uuid	Yes	NULL	Null means global grant

Primary key

- id

Foreign keys

- user_id → users.id ON DELETE CASCADE
- role_id → roles.id ON DELETE RESTRICT
- session_id → academic_sessions.id ON DELETE CASCADE

Unique constraints

- UNIQUE NULLS NOT DISTINCT (user_id, role_id, session_id)

Indexes

- btree (user_id)
- btree (role_id)
- btree (session_id) WHERE session_id IS NOT NULL

Check constraints

- None

Triggers / guards

- authorization cache invalidation after change

academic_sessions

> Defines an academic offering window in which groups, roles, milestones, and evaluations operate.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
name	varchar(80)	No	–	Example: Fall 2026
start_date	date	No	–	Inclusive start
end_date	date	No	–	Inclusive end
is_active	boolean	No	true	Operational availability

Primary key

- id

Foreign keys

- None

Unique constraints

- name

Indexes

- UNIQUE btree (name)
- btree (start_date, end_date)
- btree (is_active) WHERE is_active

Check constraints

- end_date > start_date

Triggers / guards

- optional exclusion trigger to prevent forbidden overlaps after policy approval

programs

> Stores computing program codes referenced by session offerings and project-type suitability.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
code	varchar(10)	No	–	CS, SE, IT, AI, DS, CY, or CGV
name	varchar(150)	No	–	Formal program name

Primary key

- id

Foreign keys

- None

Unique constraints

- code

Indexes

- UNIQUE btree (code)
- btree (name)

Check constraints

- code ~ '^[A-Z0-9]{2,10}$'

Triggers / guards

- None

session_programs

> Activates a program for a specific academic session.

Column	Type	Null	Default	Notes
session_id	uuid	No	–	Academic session
program_id	uuid	No	–	Program

Primary key

- (session_id, program_id)

Foreign keys

- session_id → academic_sessions.id ON DELETE CASCADE
- program_id → programs.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- PK supports session listing
- btree (program_id)

Check constraints

- None

Triggers / guards

- None

project_types

> Stores the configurable catalogue of FYP pathways and the top-level AI policy classification.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
code	varchar(40)	No	—	Stable machine code
name	varchar(150)	No	—	Framework label
description	text	No	—	Purpose and scope
suitable_programs	text[]	No	{}	Framework program codes
objectives	text	Yes	NULL	Outcome narrative
ai_policy	ai_policy	No	—	NO_AI, API_ONLY, EXTENSIVE_ALLOWED, LIMITED
is_active	boolean	No	true	Catalogue visibility

Primary key

- id

Foreign keys

- None

Unique constraints

- code

Indexes

- UNIQUE btree (code)
- GIN (suitable_programs)
- btree (is_active) WHERE is_active

Check constraints

- cardinality(suitable_programs) > 0

Triggers / guards

- validate suitable_programs against programs.code

project_groups

> Represents one student team and its project across the complete lifecycle.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_code	varchar(30)	No	generated	Public reference
title	varchar(300)	No	—	Current approved working title
project_type_id	uuid	No	—	Selected pathway
session_id	uuid	No	—	Owning academic session
status	group_status	No	FORMING	Lifecycle state
created_at	timestamptz	No	now()	Creation time
updated_at	timestamptz	No	now()	Last mutation time

Primary key

- id

Foreign keys

- project_type_id → project_types.id ON DELETE RESTRICT
- session_id → academic_sessions.id ON DELETE RESTRICT

Unique constraints

- group_code

Indexes

- UNIQUE btree (group_code)
- btree (session_id, status)
- btree (project_type_id)
- GIN trigram (title) if fuzzy search enabled

Check constraints

- btrim(title) <> ''

Triggers / guards

- generate group_code before insert
- set_updated_at before update
- validate status transition

group_memberships

> Associates students with a group and records the team leader.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Project group
user_id	uuid	No	—	Student user
member_role	member_role	No	MEMBER	LEADER or MEMBER
joined_at	timestamptz	No	now()	Membership start

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- user_id → users.id ON DELETE RESTRICT

Unique constraints

- (group_id, user_id)

Indexes

- btree (group_id)
- btree (user_id)
- UNIQUE (group_id) WHERE member_role = LEADER

Check constraints

- None

Triggers / guards

- verify user has STUDENT role for group session
- enforce at most one active group per student per session

proposals

> Stores the current proposal record for a group, including its revision number and workflow state.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	One current proposal per group
title	varchar(300)	No	—	Proposal title
abstract	text	No	—	Executive abstract
problem_statement	text	No	—	Problem definition
objectives	text	No	—	Measurable objectives
scope	text	No	—	In/out of scope
methodology	text	No	—	Approach
tools	text	Yes	NULL	Planned tools/stack
timeline_text	text	Yes	NULL	Narrative or serialized plan
document_url	text	Yes	NULL	Submitted document object key
status	proposal_status	No	DRAFT	Approval state
version	integer	No	1	Current revision
submitted_at	timestamptz	Yes	NULL	Latest submission time
created_at	timestamptz	No	now()	Creation time
updated_at	timestamptz	No	now()	Last mutation time

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE

Unique constraints

- group_id

Indexes

- UNIQUE btree (group_id)
- btree (status, submitted_at)
- GIN full-text expression over title/abstract

Check constraints

- version >= 1
- status = 'DRAFT' OR submitted_at IS NOT NULL

Triggers / guards

- increment version on resubmission
- set submitted_at on submit
- set_updated_at

proposal_reviews

> Records every reviewer decision and comment against the current proposal revision.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
proposal_id	uuid	No	—	Reviewed proposal
reviewer_id	uuid	No	—	Reviewing user
stage	varchar(40)	No	—	SUPERVISOR, COORDINATOR, or HOD
decision	review_decision	No	—	Decision code
comments	text	Yes	NULL	Required for adverse decision
created_at	timestamptz	No	now()	Decision time

Primary key

- id

Foreign keys

- proposal_id → proposals.id ON DELETE CASCADE
- reviewer_id → users.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- btree (proposal_id, created_at DESC)
- btree (reviewer_id, created_at DESC)

Check constraints

- stage IN ('SUPERVISOR','COORDINATOR','HOD')
- decision IN ('APPROVED','REJECTED','RESUBMISSION_REQUIRED','COMMENTED')
- decision IN ('APPROVED','COMMENTED') OR btrim(comments) <> ''

Triggers / guards

- snapshot proposal version into audit_logs on review

faculty_profiles

> Stores supervision-specific attributes for faculty users.

Column	Type	Null	Default	Notes
user_id	uuid	No	–	Also primary key
designation	varchar(100)	No	–	Academic designation
expertise_tags	text[]	No	{}	Searchable skill tags
max_supervision_load	integer	No	6	Configurable capacity; proposed default

Primary key

- user_id

Foreign keys

- user_id → users.id ON DELETE CASCADE

Unique constraints

- None beyond primary key

Indexes

- GIN (expertise_tags)

Check constraints

- max_supervision_load > 0

Triggers / guards

- verify faculty-eligible role

supervision_allocations

> Tracks supervisor, co-supervisor, and industry mentor assignments, including reassignment history.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Project group
supervisor_id	uuid	No	—	Assigned user
alloc_role	allocation_role	No	—	Assignment capacity
status	allocation_status	No	ACTIVE	Current state
allocated_at	timestamptz	No	now()	Start time

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- supervisor_id → users.id ON DELETE RESTRICT

Unique constraints

- (group_id, supervisor_id, alloc_role) WHERE status = ACTIVE

Indexes

- UNIQUE btree (group_id, supervisor_id, alloc_role) WHERE status = ACTIVE
- btree (supervisor_id, status)
- btree (group_id, status)

Check constraints

- None

Triggers / guards

- validate role compatibility
- enforce faculty max load for ACTIVE supervisor/co-supervisor

meetings

> Catalogues planned and completed group meetings.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Project group
title	varchar(200)	No	—	Meeting subject
scheduled_at	timestamptz	No	—	Scheduled start
duration_minutes	integer	No	—	Planned/actual duration
mode	meeting_mode	No	—	IN_PERSON or ONLINE
venue_link	text	Yes	NULL	Room or meeting URL
agenda	text	Yes	NULL	Agenda
status	meeting_status	No	SCHEDULED	Lifecycle state
created_by	uuid	No	—	Organizer
created_at	timestamptz	No	now()	Creation time

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- created_by → users.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- btree (group_id, scheduled_at DESC)
- btree (scheduled_at) WHERE status = SCHEDULED
- btree (created_by)

Check constraints

- duration_minutes BETWEEN 1 AND 1440
- mode = 'IN_PERSON' OR venue_link IS NOT NULL

Triggers / guards

- disallow completion without meeting_minutes

meeting_attendance

> Records one attendance state per meeting participant.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
meeting_id	uuid	No	–	Meeting
user_id	uuid	No	–	Participant
attendance	attendance_status	No	–	PRESENT, ABSENT, or EXCUSED

Primary key

- id

Foreign keys

- meeting_id → meetings.id ON DELETE CASCADE
- user_id → users.id ON DELETE RESTRICT

Unique constraints

- (meeting_id, user_id)

Indexes

- UNIQUE btree (meeting_id, user_id)
- btree (user_id)

Check constraints

- None

Triggers / guards

- verify participant belongs to group or active supervision allocation

meeting_minutes

> Stores the official minutes and supervisor sign-off for a completed meeting.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
meeting_id	uuid	No	—	Exactly one minutes record
notes	text	No	—	Discussion record
decisions	text	Yes	NULL	Agreed decisions
supervisor_signed_off	boolean	No	false	Formal attestation
signed_off_at	timestamptz	Yes	NULL	Attestation time
effort_hours_logged	numeric(5,2)	No	0	Meeting-related aggregate only

Primary key

- id

Foreign keys

- meeting_id → meetings.id ON DELETE CASCADE

Unique constraints

- meeting_id

Indexes

- UNIQUE btree (meeting_id)
- btree (supervisor_signed_off) WHERE NOT supervisor_signed_off

Check constraints

- effort_hours_logged >= 0
- supervisor_signed_off = false OR signed_off_at IS NOT NULL

Triggers / guards

- set signed_off_at on attestation
- freeze minutes after sign-off except controlled reopen

action_items

> Tracks accountable follow-up work arising from meetings.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
meeting_id	uuid	No	—	Source meeting
description	text	No	—	Action statement
owner_id	uuid	No	—	Responsible user
due_date	date	Yes	NULL	Due date
status	action_status	No	OPEN	OPEN, IN_PROGRESS, DONE
completed_at	timestamptz	Yes	NULL	Completion time

Primary key

- id

Foreign keys

- meeting_id → meetings.id ON DELETE CASCADE
- owner_id → users.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- btree (meeting_id)
- btree (owner_id, status, due_date)
- btree (due_date) WHERE status <> DONE

Check constraints

- status <> 'DONE' OR completed_at IS NOT NULL

Triggers / guards

- set completed_at when moved to DONE

milestones

> Defines dated checkpoints for a session, optionally limited to one project type.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
session_id	uuid	No	—	Owning session
project_type_id	uuid	Yes	NULL	Null applies to all types
title	varchar(200)	No	—	Checkpoint title
description	text	Yes	NULL	Instructions
due_date	date	No	—	Deadline
sort_order	integer	No	0	Display order

Primary key

- id

Foreign keys

- session_id → academic_sessions.id ON DELETE CASCADE
- project_type_id → project_types.id ON DELETE RESTRICT

Unique constraints

- UNIQUE NULLS NOT DISTINCT (session_id, project_type_id, title)

Indexes

- btree (session_id, due_date)
- btree (project_type_id) WHERE project_type_id IS NOT NULL

Check constraints

- sort_order >= 0

Triggers / guards

- validate due_date falls within session or approved grace period

deliverable_templates

> Defines required or optional artifacts attached to a milestone and project type.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
milestone_id	uuid	No	—	Parent milestone
project_type_id	uuid	Yes	NULL	Null means common deliverable
title	varchar(200)	No	—	Artifact name
description	text	Yes	NULL	Submission requirements
is_required	boolean	No	true	Completion requirement

Primary key

- id

Foreign keys

- milestone_id → milestones.id ON DELETE CASCADE
- project_type_id → project_types.id ON DELETE RESTRICT

Unique constraints

- UNIQUE NULLS NOT DISTINCT (milestone_id, project_type_id, title)

Indexes

- btree (milestone_id)
- btree (project_type_id) WHERE project_type_id IS NOT NULL

Check constraints

- None

Triggers / guards

- validate project_type matches milestone scope

deliverable_submissions

> Stores immutable versions of group deliverable uploads and review status.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Submitting group
template_id	uuid	No	—	Required deliverable
version	integer	No	generated	Monotonic per group/template
file_url	text	No	—	Object-storage key/URL
notes	text	Yes	NULL	Submission note
status	submission_status	No	SUBMITTED	Review state
submitted_at	timestamptz	No	now()	Submission time

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- template_id → deliverable_templates.id ON DELETE RESTRICT

Unique constraints

- (group_id, template_id, version)

Indexes

- UNIQUE btree (group_id, template_id, version)
- btree (group_id, status)
- btree (template_id, submitted_at DESC)

Check constraints

- version >= 1
- btrim(file_url) <> ''

Triggers / guards

- advisory lock + assign max(version)+1 before insert
- validate template applies to group type/session

submission_reviews

> Records reviewer decisions and comments for a specific immutable submission version.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
submission_id	uuid	No	—	Reviewed submission
reviewer_id	uuid	No	—	Reviewer
decision	submission_decision	No	—	ACCEPTED, REVISION_REQUIRED, COMMENTED
comments	text	Yes	NULL	Required for revision
created_at	timestamptz	No	now()	Review time

Primary key

- id

Foreign keys

- submission_id → deliverable_submissions.id ON DELETE CASCADE
- reviewer_id → users.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- btree (submission_id, created_at DESC)
- btree (reviewer_id, created_at DESC)

Check constraints

- decision <> 'REVISION_REQUIRED' OR btrim(comments) <> ''

Triggers / guards

- update submission status from latest decisive review

ai_declarations

> Captures a versioned AI-use declaration associated with a deliverable submission.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
submission_id	uuid	No	—	Declared submission
version	integer	No	1	Declaration revision
ai_usage_level	ai_usage_level	No	NONE	NONE, API_ONLY, EXTENSIVE, LIMITED
tools_used	text	Yes	NULL	Named tools/services
declaration_text	text	No	—	Signed narrative
declared_by	uuid	No	—	Student declarant
supervisor_attested	boolean	No	false	Supervisor verification
policy_flag	boolean	No	false	Possible policy mismatch
created_at	timestamptz	No	now()	Declaration time

Primary key

- id

Foreign keys

- submission_id → deliverable_submissions.id ON DELETE CASCADE
- declared_by → users.id ON DELETE RESTRICT

Unique constraints

- (submission_id, version)

Indexes

- UNIQUE btree (submission_id, version)
- btree (policy_flag) WHERE policy_flag
- btree (declared_by, created_at DESC)

Check constraints

- version >= 1
- ai_usage_level = 'NONE' OR tools_used IS NOT NULL

Triggers / guards

- compare against frozen project-type policy and set policy_flag
- prevent attestation by declarant

evaluation_stages

> Defines configurable evaluation stages within a session.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
session_id	uuid	No	—	Academic session
code	varchar(10)	No	—	E1 through E5 by proposed seed
name	varchar(150)	No	—	Stage label
weightage_percent	numeric(5,2)	No	—	Contribution to final result
sort_order	integer	No	—	Display/calculation order
status	evaluation_stage_status	No	DRAFT	DRAFT, OPEN, CLOSED

Primary key

- id

Foreign keys

- session_id → academic_sessions.id ON DELETE CASCADE

Unique constraints

- (session_id, code)
- (session_id, sort_order)

Indexes

- UNIQUE btree (session_id, code)
- btree (session_id, status, sort_order)

Check constraints

- weightage_percent BETWEEN 0 AND 100
- sort_order > 0

Triggers / guards

- deferred constraint trigger: session stage weightages total 100 when published/opened

rubrics

> Stores a rubric header for one evaluation stage and optionally one project type.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
evaluation_stage_id	uuid	No	—	Evaluation stage
project_type_id	uuid	Yes	NULL	Null means default rubric
name	varchar(200)	No	—	Rubric label
is_active	boolean	No	true	Selectable version

Primary key

- id

Foreign keys

- evaluation_stage_id → evaluation_stages.id ON DELETE CASCADE
- project_type_id → project_types.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- UNIQUE NULLS NOT DISTINCT (evaluation_stage_id, project_type_id) WHERE is_active
- btree (evaluation_stage_id)
- btree (project_type_id) WHERE project_type_id IS NOT NULL

Check constraints

- None

Triggers / guards

- deactivate prior rubric version when a replacement is activated; never mutate rubric used by scores

rubric_criteria

> Defines weighted scoring criteria and performance-level descriptors.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
rubric_id	uuid	No	—	Parent rubric
name	varchar(180)	No	—	Criterion label
description	text	Yes	NULL	Assessment guidance
max_marks	numeric(6,2)	No	—	Maximum raw mark
weightage	numeric(5,2)	No	—	Share within rubric
sort_order	integer	No	—	Display order
level_descriptors	jsonb	No	{}	Structured performance bands

Primary key

- id

Foreign keys

- rubric_id → rubrics.id ON DELETE CASCADE

Unique constraints

- (rubric_id, sort_order)
- (rubric_id, name)

Indexes

- btree (rubric_id, sort_order)
- GIN (level_descriptors) only if descriptor search is required

Check constraints

- max_marks > 0
- weightage BETWEEN 0 AND 100
- sort_order > 0
- jsonb_typeof(level_descriptors) = 'object'

Triggers / guards

- deferred constraint trigger: criterion weightages total 100 per active rubric

evaluations

> Schedules and tracks one group assessment at one evaluation stage.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Evaluated group
stage_id	uuid	No	—	Evaluation stage
scheduled_at	timestamptz	Yes	NULL	Panel time
venue	text	Yes	NULL	Room or online location
status	evaluation_status	No	SCHEDULED	Workflow state

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- stage_id → evaluation_stages.id ON DELETE RESTRICT

Unique constraints

- (group_id, stage_id)

Indexes

- UNIQUE btree (group_id, stage_id)
- btree (stage_id, status)
- btree (scheduled_at) WHERE status = SCHEDULED

Check constraints

- status = 'SCHEDULED' OR scheduled_at IS NOT NULL

Triggers / guards

- verify group session equals stage session
- freeze applicable rubric snapshot at scoring start

evaluation_panels

> Assigns evaluators and panel functions to an evaluation.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
evaluation_id	uuid	No	—	Evaluation
evaluator_id	uuid	No	—	Evaluator user
panel_role	panel_role	No	—	INTERNAL, EXTERNAL, or CHAIR

Primary key

- id

Foreign keys

- evaluation_id → evaluations.id ON DELETE CASCADE
- evaluator_id → users.id ON DELETE RESTRICT

Unique constraints

- (evaluation_id, evaluator_id)

Indexes

- UNIQUE btree (evaluation_id, evaluator_id)
- btree (evaluator_id)
- UNIQUE (evaluation_id) WHERE panel_role = CHAIR

Check constraints

- None

Triggers / guards

- verify evaluator role and conflict-of-interest rule

evaluation_scores

> Stores one evaluator mark per criterion for a group evaluation.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
evaluation_id	uuid	No	—	Evaluation
criterion_id	uuid	No	—	Rubric criterion
evaluator_id	uuid	No	—	Scoring evaluator
marks_obtained	numeric(6,2)	No	—	Raw criterion mark
comments	text	Yes	NULL	Evaluator rationale
created_at	timestamptz	No	now()	Score time

Primary key

- id

Foreign keys

- evaluation_id → evaluations.id ON DELETE CASCADE
- criterion_id → rubric_criteria.id ON DELETE RESTRICT
- evaluator_id → users.id ON DELETE RESTRICT

Unique constraints

- (evaluation_id, criterion_id, evaluator_id)

Indexes

- UNIQUE btree (evaluation_id, criterion_id, evaluator_id)
- btree (evaluator_id, created_at DESC)
- btree (criterion_id)

Check constraints

- marks_obtained >= 0

Triggers / guards

- BEFORE INSERT/UPDATE: marks_obtained <= rubric_criteria.max_marks
- verify evaluator is on evaluation panel
- block changes after MODERATED/PUBLISHED except controlled reopen

notifications

> Stores per-user in-portal notifications with optional entity deep links.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
user_id	uuid	No	–	Recipient
title	varchar(200)	No	–	Short heading
body	text	No	–	Message body
notif_type	varchar(40)	No	–	Application event code
related_entity_type	varchar(60)	Yes	NULL	Polymorphic entity label
related_entity_id	uuid	Yes	NULL	Related row identifier
is_read	boolean	No	false	Read state
created_at	timestamptz	No	now()	Creation time

Primary key

- id

Foreign keys

- user_id → users.id ON DELETE CASCADE

Unique constraints

- None beyond primary key

Indexes

- btree (user_id, is_read, created_at DESC)
- btree (related_entity_type, related_entity_id) WHERE related_entity_id IS NOT NULL

Check constraints

- (related_entity_type IS NULL) = (related_entity_id IS NULL)

Triggers / guards

- None

announcements

> Stores institution/session announcements targeted at role audiences.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
session_id	uuid	Yes	NULL	Null means institution-wide
title	varchar(200)	No	—	Announcement heading
body	text	No	—	Announcement content
audience_roles	text[]	No	{}	Target role codes
published_by	uuid	No	—	Publisher
published_at	timestamptz	Yes	NULL	Null while draft

Primary key

- id

Foreign keys

- session_id → academic_sessions.id ON DELETE CASCADE
- published_by → users.id ON DELETE RESTRICT

Unique constraints

- None beyond primary key

Indexes

- btree (session_id, published_at DESC)
- GIN (audience_roles)
- btree (published_at DESC) WHERE published_at IS NOT NULL

Check constraints

- cardinality(audience_roles) > 0

Triggers / guards

- validate audience role codes against roles.name

audit_logs

> Stores append-only evidence of security-sensitive and academic record changes.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
actor_id	uuid	Yes	NULL	Null for trusted system actor
action	varchar(80)	No	—	CREATE, UPDATE, DELETE, LOGIN, PUBLISH, etc.
entity_type	varchar(80)	No	—	Logical table/entity
entity_id	uuid	Yes	NULL	Affected row
old_values	jsonb	Yes	NULL	Redacted prior values
new_values	jsonb	Yes	NULL	Redacted new values
ip_address	inet	Yes	NULL	Request source
created_at	timestamptz	No	now()	Event time

Primary key

- id

Foreign keys

- actor_id → users.id ON DELETE SET NULL

Unique constraints

- None beyond primary key

Indexes

- btree (entity_type, entity_id, created_at DESC)
- btree (actor_id, created_at DESC)
- BRIN (created_at) for large append-only history

Check constraints

- action <> ''
- entity_type <> ''

Triggers / guards

- deny UPDATE/DELETE to application roles
- partition by time when volume threshold is reached

effort_entries

> Recommended core extension: records per-student work evidence needed to evaluate 9 hours/week and 300 total hours.

Column	Type	Null	Default	Notes
id	uuid	No	gen_random_uuid()	Primary identifier
group_id	uuid	No	—	Project group
student_id	uuid	No	—	Student
work_date	date	No	—	Date worked
hours	numeric(4,2)	No	—	Effort amount
category	varchar(40)	No	—	RESEARCH, DESIGN, BUILD, TEST, DOCUMENT, MEETING, OTHER
description	text	No	—	Evidence narrative
evidence_url	text	Yes	NULL	Optional supporting artifact
verified_by	uuid	Yes	NULL	Supervisor verifier
verified_at	timestamptz	Yes	NULL	Verification time
created_at	timestamptz	No	now()	Entry time

Primary key

- id

Foreign keys

- group_id → project_groups.id ON DELETE CASCADE
- student_id → users.id ON DELETE RESTRICT
- verified_by → users.id ON DELETE SET NULL

Unique constraints

- None beyond primary key

Indexes

- btree (student_id, work_date)
- btree (group_id, work_date)
- btree (verified_by) WHERE verified_by IS NOT NULL

Check constraints

- hours > 0 AND hours <= 24
- (verified_by IS NULL) = (verified_at IS NULL)

Triggers / guards

- verify student membership and verifier supervision relationship
- prevent edits after verification except audited reopen

Enum registry and controlled values

Type	Allowed values
ai_policy	NO_AI, API_ONLY, EXTENSIVE_ALLOWED, LIMITED
group_status	FORMING, PROPOSED, APPROVED, IN_PROGRESS, EVALUATION, COMPLETED, ARCHIVED
member_role	LEADER, MEMBER
proposal_status	DRAFT, SUBMITTED, UNDER_SUPERVISOR_REVIEW, UNDER_COORDINATOR_REVIEW, APPROVED, REJECTED, RESUBMISSION_REQUIRED
review_decision	APPROVED, REJECTED, RESUBMISSION_REQUIRED, COMMENTED
allocation_role	SUPERVISOR, CO_SUPERVISOR, INDUSTRY_MENTOR
allocation_status	ACTIVE, REASSIGNED, ENDED
meeting_mode	IN_PERSON, ONLINE
meeting_status	SCHEDULED, COMPLETED, CANCELLED
attendance_status	PRESENT, ABSENT, EXCUSED
action_status	OPEN, IN_PROGRESS, DONE
submission_status	SUBMITTED, UNDER_REVIEW, REVISION_REQUIRED, ACCEPTED
submission_decision	ACCEPTED, REVISION_REQUIRED, COMMENTED
ai_usage_level	NONE, API_ONLY, EXTENSIVE, LIMITED
evaluation_stage_status	DRAFT, OPEN, CLOSED
evaluation_status	SCHEDULED, IN_PROGRESS, SCORED, MODERATED, PUBLISHED
panel_role	INTERNAL, EXTERNAL, CHAIR

Migration policy

Adding an enum value is backward-compatible only after all application versions tolerate it. Renaming or removing a value requires a staged migration: add replacement, dual-read, backfill, switch writers, verify, then retire. If frequent policy changes are expected, use lookup tables with stable codes rather than PostgreSQL enums.

> **Roles are not an enum.** Role rows are seeded catalogue data so descriptions and permission mappings can evolve. Their role names remain constrained to the approved list in this version.

Data integrity and cross-table constraints

Rule	Enforcement
Session dates ordered	academic_sessions CHECK
One leader per group	Partial unique index
Student in one active group/session	Constraint trigger across memberships/groups
Supervisor capacity	Deferred trigger with locking
Proposal review matches current version	Audit snapshot; add proposal_version column to reviews if full history required
Template applies to group	Submission trigger
Submission versions monotonic	Advisory lock + trigger
AI use matches policy	Declaration trigger sets flag; human resolves
Stage weights total 100	Deferred constraint trigger at activation
Rubric criteria total 100	Deferred constraint trigger at activation
Score not above maximum	Score trigger joins criterion
Evaluator belongs to panel	Score trigger
Evaluation/group session match	Evaluation trigger
Signed/published records frozen	Mutation guard trigger
Effort actor relationships valid	Effort trigger

Constraint timing

Simple row rules use CHECK constraints. Rules that require another row use immediate triggers. Rules that require a set total or multiple inserts use deferrable constraint triggers evaluated at transaction commit. Triggers raise stable SQLSTATE/application error codes so the API can translate failures predictably.

Group code generation and workflow locking

Format

```
FYP-<session-token>-<sequence>
Example: FYP-F26-001
```

The displayed format is configurable. A recommended token maps “Fall 2026” to `F26`. Never derive uniqueness from row count.

Safe allocation algorithm

1. Lock a per-session counter row, or call a PostgreSQL sequence dedicated to the session.
2. Increment atomically inside the group-creation transaction.
3. Format the number with a minimum width of three digits without imposing a maximum.
4. Insert the group under the unique group_code constraint.
5. Write an audit event containing session, generated code, and actor.

State-change locking

Transitions that depend on completion of related records use `SELECT ... FOR UPDATE` on the parent group, proposal, submission, or evaluation. This prevents two requests from approving/reopening/publishing the same record concurrently.

> **Do not recycle codes.** Cancelled or archived groups keep their public code. The sequence may contain gaps; gaps are acceptable and must not be filled by hand.

State transitions are explicit, not arbitrary updates

Object	Allowed path
Group	FORMING → PROPOSED → APPROVED → IN_PROGRESS → EVALUATION → COMPLETED → ARCHIVED
Proposal	DRAFT → SUBMITTED → UNDER_SUPERVISOR_REVIEW → UNDER_COORDINATOR_REVIEW → APPROVED
Proposal exception	Review states → RESUBMISSION_REQUIRED → SUBMITTED; review states → REJECTED
Submission	SUBMITTED → UNDER_REVIEW → ACCEPTED or REVISION_REQUIRED
Meeting	SCHEDULED → COMPLETED or CANCELLED
Evaluation	SCHEDULED → IN_PROGRESS → SCORED → MODERATED → PUBLISHED
Action item	OPEN → IN_PROGRESS → DONE; DONE may reopen only with audit
Allocation	ACTIVE → REASSIGNED or ENDED

Backward transitions are rejected unless a named privileged operation records a reason and writes an audit event. Project group advancement checks prerequisites: approved proposal before IN_PROGRESS, required submissions before EVALUATION, all mandatory stages moderated before COMPLETED, and published result before ARCHIVED if policy requires it.

Indexes support ownership, queues, and history

Index principles

- Index every foreign key to avoid lock amplification and slow joins.
- Lead composite indexes with equality filters, then status/date ordering used by queues.
- Use partial indexes for active, unread, pending, scheduled, or flagged subsets.
- Use GIN only where array, JSON, or full-text queries are real product requirements.
- Use BRIN for large append-only time series such as audit_logs after row volume justifies it.
- Review index usage quarterly; remove redundant indexes only after production evidence.

Representative query/index pairs

Query	Supporting index
Unread notifications by user	notifications(user_id,is_read,created_at DESC)
Upcoming meetings for group	meetings(group_id,scheduled_at DESC)
Active supervision load	supervision_allocations(supervisor_id,status)
Pending submissions for reviewer	submission status + template/stage joins
Evaluation schedule	evaluations(scheduled_at) WHERE SCHEDULED
Entity audit history	audit_logs(entity_type,entity_id,created_at DESC)
Student weekly effort	effort_entries(student_id,work_date)

Transactional boundaries preserve academic truth

Operation	Single-transaction work
Create group	counter/code, group, leader membership, audit
Submit proposal	validate membership/state, increment version, timestamp, group state, notify, audit
Allocate supervisor	capacity lock, end old allocation if needed, insert allocation, notify, audit
Complete meeting	minutes, attendance, action items, status, effort link, audit
Submit deliverable	version lock, row insert, AI declaration requirement, notify, audit
Record scores	panel check, rubric check, upsert evaluator marks, completeness check
Moderate evaluation	aggregate, variance checks, status transition, audit
Publish result	recalculate, snapshot or signed view, status, notifications, audit

Isolation and retries

Use `READ COMMITTED` for ordinary CRUD with explicit row locks. Use `SERIALIZABLE` or advisory locks for version/counter allocation and publish operations when invariants span multiple rows. Retry serialization failures with bounded jitter in the service layer; never retry validation failures.

Idempotency

External requests that may repeat should include an idempotency key stored in a dedicated request ledger or outbox. Database notifications should be written to an outbox in the same transaction, then delivered asynchronously.

Seed the framework catalogue with provenance

Code	Project type	Programs	Encoded AI policy
PRODUCT_DEV	Product Development Project	CS, SE, IT	NO_AI; API exception note
ENTREPRENEURIAL	Entrepreneurial Project	CS, SE, IT	EXTENSIVE_ALLOWED
INDUSTRY_CONTINUATION	Continuation Project (Existing product based Project)	CS, SE, IT, CY, DS, AI, CGV	LIMITED; APIs/features allowed; docs restricted
RESEARCH	Research Project	CS, AI, CY, DS	NO_AI; API exception note
IOT_EMBEDDED	IoT & Embedded Systems Project	CS, SE, IT	NO_AI; API exception note
CASE_STUDY	Case Study Project	IT, CY, DS	LIMITED; documentation prohibited
CONSULT_AI_DA	Consulting Project (AI/Data Analytics)	AI, DS	LIMITED; documentation prohibited
CONSULT_CYBER	Consulting Project (Cybersecurity)	CY, IT	LIMITED; documentation prohibited
CONSULT_CLOUD_DEVOPS	Consulting Project (Cloud & DevOps)	IT, CS, SE	LIMITED; documentation prohibited
CONSULT_DIGITAL_TX	Consulting Project (Digital Transformation)	IT, CS, SE	LIMITED; documentation prohibited

Encoding limitation. One ai_policy enum cannot express “API use allowed but documentation use prohibited.” Preserve the exact framework wording in description/objectives or add project_type_ai_rules(scope, allowance, rule_text). The encoded value is a routing aid, not the complete policy.

Seed roles, permissions, and configurable stages

Roles

Seed exactly: SUPER_ADMIN, FYP_COORDINATOR, HOD, SUPERVISOR, CO_SUPERVISOR, STUDENT, INTERNAL_EVALUATOR, EXTERNAL_EXAMINER, INDUSTRY_PARTNER.

Permission families

Family	Examples
Users/roles	users.read, users.manage, roles.assign
Groups	groups.create, groups.approve, groups.archive
Proposals	proposals.submit, proposals.review, proposals.approve
Supervision	allocations.manage, meetings.sign_off
Deliverables	submissions.submit, submissions.review
Evaluation	panels.manage, scores.write, evaluations.moderate, results.publish
Policy/config	sessions.manage, milestones.manage, rubrics.manage
Audit/reporting	audit.read, reports.export

Proposed default evaluation stages

Code	Working name	Weight
E1	Proposal evaluation	10%
E2	Mid-term evaluation	20%
E3	Continuous assessment	10%
E4	Final demonstration	30%
E5	Report and viva	30%

> **Configurable.** The names, stage count, dates, rubrics, and proposed 10/20/10/30/30 weightages require academic approval per session. The database only enforces that active weights total 100%.

Migrations are forward-only, rehearsed, and reversible by design

Migration sequence

1. Enable extensions and create enums/lookups.
2. Create identity and academic catalogue tables.
3. Create project, membership, proposal, supervision, meeting, deliverable, AI, evaluation, communication, audit, and effort tables.
4. Create indexes and shared timestamp/audit functions.
5. Create integrity triggers and row-level security policies.
6. Seed programs, roles, permissions, role mappings, and ten framework project types.
7. Create a session only after committee configuration is approved.
8. Run verification queries and record migration checksum.

Zero-downtime change pattern

Add nullable column → deploy dual-write/read-compatible application → backfill in bounded batches → validate → add NOT NULL/check as NOT VALID then VALIDATE → switch reads → remove old column in a later release.

Seed provenance

Every policy seed should include a migration identifier and source version in a seed ledger. Re-running seeds must be idempotent with `INSERT ... ON CONFLICT ... DO UPDATE` only for explicitly mutable fields. Never overwrite historical session configuration.

Reporting views derive facts without weakening controls

View/materialization	Purpose
v_group_progress	Latest milestone/submission state by group
v_supervision_load	Active group count against faculty capacity
v_student_effort_weekly	Weekly and cumulative verified hours
v_pending_reviews	Proposal and deliverable review queues
v_evaluation_completeness	Missing panel/criterion scores
v_group_stage_results	Normalized panel aggregate per stage
v_final_results	Weighted total across stages
v_ai_policy_flags	Unresolved declaration mismatches
v_meeting_compliance	Frequency, attendance, unsigned minutes
mv_session_dashboard	Pre-aggregated coordinator dashboard

Result formula baseline

```
criterion_ratio = marks_obtained / max_marks
criterion_weighted = criterion_ratio × criterion.weightage
panel_stage_score = Σ criterion_weighted
stage_score = approved panel aggregation(panel_stage_score)
final_score = Σ(stage_score × stage.weightage_percent / 100)
```

Panel aggregation (mean, median, chair decision, outlier rule) is an open academic decision. The chosen algorithm and rounding precision must be versioned and included in any published result snapshot.

Backup, recovery, retention, and archival

Operational baseline

Control	Design requirement
Backups	Encrypted full backups plus continuous WAL/PITR
Recovery tests	Scheduled restore drills with evidence
RPO/RTO	To be approved before production
Monitoring	Connections, locks, replication lag, disk, slow queries, failed jobs
Archival	Archive sessions logically before any cold-storage move
Retention	Policy required by data class; no silent purge
Exports	Authorized, watermarked where needed, and audited
Secrets	Managed outside database and source repository

Archival strategy

Set project groups to ARCHIVED after session closure and make records read-only. Keep catalogue records referenced by history. If old audit/event data moves to partitions or cold storage, preserve queryable chain-of-custody metadata and test restores.

Disaster recovery acceptance

A backup is not accepted until a clean environment can restore the schema, extensions, seed catalogue, file references, and representative academic history, and the application can verify result totals.

> **Open decision.** RSCI must approve retention periods for profiles, IP addresses, meeting records, submissions, evaluations, audit logs, and external examiner access.

Database tests target invariants, not only CRUD

Test class	Required coverage
Schema	Types, defaults, nullability, keys, extension availability
Constraints	Every CHECK, unique rule, and FK negative case
Triggers	Cross-session rejection, score cap, totals, freezes, version allocation
Authorization	Allow/deny matrix for every role and session scope
Concurrency	Group code, submission version, supervisor capacity, publish race
Calculation	Rubric normalization, panel aggregation, stage weighting, rounding
Migration	Fresh install, upgrade from previous, seeded idempotency, rollback plan
Performance	Dashboard, queues, audit history at expected high-volume scale
Backup/restore	Point-in-time restore and result checksum
Security	Audit redaction, no direct audit mutation, least-privilege grants

Acceptance thresholds

Performance targets must come from the SRS and production sizing. Before launch, capture `EXPLAIN (ANALYZE, BUFFERS)` for the ten most important queries with realistic data volume; reject sequential scans on large queue/history tables unless intentional.

Starter DDL - extensions, types, and identity

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;

CREATE TYPE ai_policy AS ENUM
  ('NO_AI','API_ONLY','EXTENSIVE_ALLOWED','LIMITED');
CREATE TYPE group_status AS ENUM
  ('FORMING','PROPOSED','APPROVED','IN_PROGRESS',
   'EVALUATION','COMPLETED','ARCHIVED');

CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email citext NOT NULL UNIQUE,
  password_hash text,
  full_name varchar(200) NOT NULL,
  phone varchar(32), avatar_url text,
  is_active boolean NOT NULL DEFAULT true,
  last_login_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (email <> '')
);

CREATE TABLE roles (
  id smallserial PRIMARY KEY,
  name varchar(40) NOT NULL UNIQUE,
  description text
);
CREATE TABLE permissions (
  id smallserial PRIMARY KEY,
  code varchar(100) NOT NULL UNIQUE,
  description text NOT NULL,
  CHECK (code ~ '^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$')
);
```

The complete migration should create every enum and table exactly as specified, then apply foreign keys, indexes, grants, row-level security, triggers, and seed data in separately reviewable steps.

Starter DDL - academic catalogue and group

```
CREATE TABLE academic_sessions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name varchar(80) NOT NULL UNIQUE,
  start_date date NOT NULL,
  end_date date NOT NULL,
  is_active boolean NOT NULL DEFAULT true,
  CHECK (end_date > start_date)
);

CREATE TABLE project_types (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code varchar(40) NOT NULL UNIQUE,
  name varchar(150) NOT NULL,
  description text NOT NULL,
  suitable_programs text[] NOT NULL,
  objectives text,
  ai_policy ai_policy NOT NULL,
  is_active boolean NOT NULL DEFAULT true,
  CHECK (cardinality(suitable_programs) > 0)
);

CREATE TABLE project_groups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  group_code varchar(30) NOT NULL UNIQUE,
  title varchar(300) NOT NULL CHECK (btrim(title) <> ''),
  project_type_id uuid NOT NULL REFERENCES project_types(id),
  session_id uuid NOT NULL REFERENCES academic_sessions(id),
  status group_status NOT NULL DEFAULT 'FORMING',
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_groups_session_status
  ON project_groups(session_id, status);
```

Starter DDL - score maximum trigger

```
CREATE OR REPLACE FUNCTION enforce_score_max()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE allowed numeric(6,2);
BEGIN
  SELECT max_marks INTO allowed
  FROM rubric_criteria WHERE id = NEW.criterion_id;
  IF allowed IS NULL THEN
    RAISE EXCEPTION USING ERRCODE = '23503',
      MESSAGE = 'Unknown rubric criterion';
  END IF;
  IF NEW.marks_obtained < 0 OR NEW.marks_obtained > allowed THEN
    RAISE EXCEPTION USING ERRCODE = '23514',
      MESSAGE = 'Score outside criterion range';
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_score_max
BEFORE INSERT OR UPDATE OF marks_obtained, criterion_id
ON evaluation_scores
FOR EACH ROW EXECUTE FUNCTION enforce_score_max();
```

Additional trigger functions required

- set_updated_at()
- validate_group_status_transition()
- assign_group_code()
- assign_submission_version()
- validate_student_membership()
- validate_supervision_capacity()
- validate_submission_applicability()
- flag_ai_policy_mismatch()
- validate_stage_weight_total()
- validate_rubric_weight_total()
- freeze_signed_or_published_record()
- write_audit_event()

Framework-to-schema traceability

Framework need	Primary storage
Ten project types and programs	project_types, programs
Common and type-specific deliverables	milestones, deliverable_templates
Project proposal	proposals, proposal_reviews
Planning/timeline	proposals.timeline_text, milestones
Implementation/supporting artifacts	deliverable_submissions
Meeting catalogue and supervision	meetings, minutes, attendance, allocations
Evaluation and oral defense	evaluation_stages, evaluations, panels, scores
Rubrics and criteria	rubrics, rubric_criteria
AI restrictions/declarations	project_types.ai_policy, ai_declarations
9 hours/week and 300 hours	effort_entries; meeting aggregate is supplemental
Technical/user documentation	deliverable templates and submissions
Reflection/future work	configured deliverable template
Traceable administration	user_roles, permissions, audit_logs

Requirements not safely represented by one field

AI restrictions vary by activity (core functionality, API use, new feature development, and documentation). The baseline enum supports routing and flags, but production should add normalized rule scope or preserve a structured policy snapshot on each project/session.

Open decisions requiring academic or governance approval

Decision	Why it matters
Group size and leadership rules	Membership constraints and UI
Supervisor/co-supervisor load	Allocation trigger and capacity
Session overlap and grace periods	Date constraints
Proposal approval chain	Required stages and permissions
Meeting frequency/sign-off deadline	Compliance reporting
Effort evidence and verification	300-hour auditability
Evaluation stages/weightages	Final-result calculation
Panel aggregation/outlier handling	Fairness and reproducibility
Rubric scale/rounding	Score precision
Result moderation/appeal/reopen	State transitions and audit
AI rule granularity	Policy validation
File retention/version limits	Storage and archival
Personal-data retention/anonymization	Privacy and audit
Backup RPO/RTO	Infrastructure design
External access expiry	Security

These decisions must be stored as versioned session configuration or implemented in approved policy tables, not hard-coded in application branches.

Implementation readiness checklist

Before development

- Approve open policy decisions and role/permission matrix.
- Confirm SRS identifiers and API state transitions match this schema.
- Choose enum versus lookup-table strategy for values expected to change.
- Add normalized scoped AI policy rules or approve exact text snapshot approach.
- Confirm result publication snapshot requirement and appeals model.

Before production

- All migrations pass clean install and upgrade tests.
- All foreign keys, unique rules, partial indexes, and trigger tests pass.
- Row-level security deny tests pass for every role and cross-session case.
- Seed data matches the approved framework and session configuration.
- Restore drill meets approved RPO/RTO and result checksums match.
- Audit logs redact secrets and cannot be changed by the application role.
- Performance plans are captured against realistic volume.
- Monitoring, rotation, archival, and incident runbooks are approved.

> **Definition of done.** The database is production-ready only when academic policy, calculation rules, security tests, migration rehearsal, and restore evidence are approved together.

Glossary

Term	Meaning
FYP	Final Year Project
RSCI	Riphah School of Computing & Innovation
Session	Academic offering such as Fall 2026
Project type	Framework pathway defining outcomes, artifacts, and AI policy
Milestone	Dated checkpoint
Deliverable template	Required artifact definition
Submission	Immutable version of a group artifact
Rubric	Set of weighted evaluation criteria
Criterion	One scored dimension with maximum and performance bands
Moderation	Controlled review/adjustment before publication
RLS	PostgreSQL row-level security
PITR	Point-in-time recovery
RPO	Recovery point objective
RTO	Recovery time objective
DDL	Data definition language

Source note

Project type names, suitable programs, common deliverable expectations, AI-use statements, and the 9-hours-per-week / 300-hour expectation are grounded in the supplied *RSCI FYP Framework v1.0*. Technical controls and proposed defaults in this document are implementation recommendations pending approval.